Practical Task 7.1

(Pass Task)

Submission deadline: Monday, May 4
Discussion deadline: Friday, May 22

Instructions

In this task, answer all the following questions and complement each answer with a detailed explanation.

1. Draw a series of figures demonstrating the insertion of the values

   18, 9, 1, 6, 5, 7, 25, 40, 10, 4, 16

   into an initially ***empty*** AVL tree. Insert the values in the order they appear in the given sequence. You must:

   - Show the resulting AVL tree immediately before and after each insertion step that causes the tree's rebalancing.
   - Calculate the ***balance degree*** (as the difference between the heights of the left and the right subtrees rooted at a node) and label each node of the AVL tree before and after the necessary rebalancing.
   - Clearly indicate the node(s) at which the rotation is performed. Here, let v be the first node you encounter in going up from the newly added node, say z, towards the root of the AVL tree T such that v is unbalanced. Then, let x be the child of v such that x is the ancestor of z. Determine whether the subtree of T rooted at v is ***right*** or ***left heavy***. Similarly, indicate whether the subtree of T rooted at x is ***right*** or ***left heavy***.
   - Performing a ***single*** or a ***double rotation*** as a rebalancing (repairing) operation on the AVL tree, specify the type of the rotation that you apply: ***Single Left Rotation***, ***Single Right Rotation***, ***Left-Right Rotation***, or ***Right-Left Rotation***.
   - Each time a new value is added, ensure that both the ***Binary Search Tree Property*** and the ***Height-Balance (AVL Tree) Property*** are maintained.

2. Now, draw a series of figures showing the deletion of the values:

   18, 10, 7, 25, 40, 9, 16, 4, 5, 1, 6

   from the AVL tree built in the previous part of the task. Delete the values in the order they appear in the given sequence. Draw the AVL tree after each deletion and rotation (if any) and complement each of the figures with all the aforementioned details.

   When you delete a node with two children, you must always replace it with ***'the largest element smaller than the key of the deleted node'*** (as opposed to the other possible rule: 'the smallest element larger than the key of the deleted node'). **Following this rule is important** to let us check your solution quickly.

3. Develop a linear $O(n)$ time algorithm that accepts an arbitrary binary search tree as an input and creates a **balanced** binary search tree whose height is $O(\log n)$. As the answer to this question, provide its pseudocode and description. Explain why you believe that your algorithm takes a linear time.

Further Notes

- The lecture recordings for weeks 6 and 7 should serve as a good starting point to understand AVL- and binary search trees.

- You will learn how to complete this task by reading Sections 11.1.1, 11.1.2, and 11.3.1 of the course book "Data Structures and Algorithms in Java" by Michael T. Goodrich, Irvine Roberto Tamassia, and Michael H. Goldwasser (2014). You may access the book on-line for free from the reading list application in CloudDeakin available in Content → Reading List → Course Book: Data structures and algorithms in Java.

Submission Instructions and Marking Process

To get your task completed, you must finish the following steps strictly on time.

- **Submit** your answers as a single PDF report via OnTrack submission system.
- Once your answers are accepted by your tutor, you will be invited to **proceed with their discussion through a face-to-face interview**. Specifically, you will need to meet with the tutor to demonstrate and discuss your answers in one of the dedicated practical sessions (run online via MS Teams for online students and on-campus for students who selected to join classes at Burwood\Geelong). Please, come prepared so that the class time is used efficiently and fairly for all students in it. Be on time with respect to the specified discussion deadline.
  You will also need to **answer all additional questions** that your tutor may ask you. Questions will cover the lecture notes; so, attending (or watching) the lectures should help you with this **compulsory** discussion part. You should start the discussion as soon as possible as if your answers are wrong, you may have to pass another round, still before the deadline. Use available attempts properly.

Note that we will not accept your solution after **the submission deadline** and will not discuss it after **the discussion deadline**. If you fail one of the deadlines, you fail the task, and this reduces the chance to pass the unit. Unless extended for all students, the deadlines are strict to guarantee smooth and on-time work throughout the unit.

Remember that this is your responsibility to keep track of your progress in the unit that includes checking which tasks have been marked as completed in the OnTrack system by your marking tutor, and which are still to be finalised. When grading your achievements at the end of the unit, we will solely rely on the records of the OnTrack system and feedback provided by your tutor about your overall progress and the quality of your solutions.